



Assignment 09: Typological Differences Between Linguistic Areas

handed out: 4 July, 03:00
to be submitted by: 10 July, 22:00

In this task, we will get back to the GramBank dataset, but instead of playing around with a limited number of features, we will approach the question whether some (combinations of) grammatical features are so characteristic of each macroarea (e.g. Eurasia or Africa) that we can use them to predict which macroarea a language will be from based on its grammatical features. For this task, all the data you need is already preprocessed into a single file `grambank.csv`.

Task 1: Data Preparation and Reshaping

- Load the table into a Pandas dataframe, and familiarise yourself with the contents. There is data covering 195 features across almost 2500 languages, and there are null values due to gaps in the grammars from which the information was extracted. For convenience, the features have unified names of shape `GBXXX`. The final column contains the macroareas which we will try to predict based on the feature values.
- Generate a binarised version of the dataset using the `pandas.get_dummies()` function. Do not change the `drop_first` or `dummy_na` arguments from their defaults, the intended behaviour is that there be separate columns for the presence and absence of each binary feature, and a missing value should be indicated by a zero in both columns.
- Prepare the feature matrix `X` and the label vector `y` in formats that the Scikit-Learn classifiers will handle. This involves casting the NumPy array to integers (the default `uint8` will not work!), and converting the labels into an array of strings (not objects!) which can then be converted to the integer IDs that will be required (Hint: use `sklearn.preprocessing.LabelEncoder`).
- Prepare train and test splits `X_train`, `y_train`, `X_test`, `y_test` by randomly selecting 80% of languages as training data and 20% as test data. (Hint: use `sklearn.model_selection.train_test_split`)

Task 2: Exploring the Data with a Decision Tree

- Fit a decision tree classifier with the default settings, and use the decision tree that you have trained in this way in order to predict the labels on the test data.
- Use `sklearn.metrics.zero_one_loss` in order to compute the accuracy of the predictions (Hint: make sure you understand the connection of the function to accuracy!). How well does your single decision tree work? Are you happy with the performance? Can you show what is going on?
- How many leaves does your decision tree have, and what are the most important features in the decision tree? Use the Grambank webpage to find out what these three variables are about.

Task 3: A Random Forest Classifier

- Fit a random forest classifier with 100 estimators to the training data (using the default settings of all other parameters), and compute the predictions on the test data.
- Compute the accuracy again, and compare it to the accuracy of the decision tree you inferred in the previous task. Do you have any idea what might cause the difference you are observing?
- In order to understand which language areas show grammatical profiles that are too difficult to differentiate, we can take a look at the confusion matrix. Create and visualise the confusion matrix based on the code from the slides about the multinomial Naive Bayes example, but use the `normalize` argument of the confusion matrix method to normalise the entries with respect to the true labels. Inspecting the results, which pairs of macroareas appear to be the hardest to distinguish?

Task 4: Support Vector Machine

- a) Fit a linear Support Vector Machine to the same dataset, using the recommended setting for the fudge factor. How does performance compare with the previous two approaches?
- b) Try out what happens with the default RBF kernel. Does performance improve any further?
- c) Create the truth-normalised confusion matrix for the SVC with the RBF kernel. Are the most confusable pairs of macroareas similar? Do these considerations lead to any idea about how we could improve performance further?

Task 5: k-Neighbour Clustering

- a) Fit a k-neighbours classifier to the same dataset. Which metric does it use by default, and how is it different from the Euclidean distance?
- b) Compute the accuracy in the usual way. How does k-neighbour clustering perform compared to the previous models? Would you have expected this?
- c) Experiment with different numbers of neighbours to take into account. What do you observe? What appears to be a good range for this problem?
- d) Create the truth-normalised confusion matrix for your choice of `n_neighbors`. Do you notice anything different to the previous two confusion matrices?